In [3]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import xgboost as xgb
```

In [17]:
```
import matplotlib.pyplot as plt
```

In [5]:
```
# Load data
train = pd.read_parquet("train_data.parquet")
test = pd.read_parquet("test_data.parquet")

transactions = pd.read_parquet("add_trans.parquet")
events = pd.read_parquet("add_event.parquet")
print("Data loaded")
```

```
✅ Data loaded
```

In [6]:
```
offer_meta = pd.read_parquet("offer_metadata.parquet")
```

In [7]:
```
# Convert IDs to consistent type
for df in [train, test]:
    df['id2'] = df['id2'].astype(str)
    df['id3'] = df['id3'].astype(str)

transactions['id2'] = transactions['id2'].astype(str)
offer_meta['id3'] = offer_meta['id3'].astype(str)
events['id2'] = events['id2'].astype(str)

# Parse datetime columns
transactions["f370"] = pd.to_datetime(transactions["f370"], errors='coerc
events["id4"] = pd.to_datetime(events["id4"], errors='coerce')
offer_meta["id12"] = pd.to_datetime(offer_meta["id12"], errors='coerce')
offer_meta["id13"] = pd.to_datetime(offer_meta["id13"], errors='coerce')
```

In [8]:
```
transactions["f367"] = pd.to_numeric(transactions["f367"], errors='coerce

txn_agg = transactions.groupby('id2').agg(
    txn_total_spend=('f367', 'sum'),
    txn_avg_spend=('f367', 'mean'),
    txn_count=('f367', 'count'),
    txn_unique_products=('f368', 'nunique'),
    txn_freq_days=('f370', lambda x: (x.max() - x.min()).days if len(x) >
).reset_index()

train = train.merge(txn_agg, on="id2", how="left")
test  = test.merge(txn_agg, on="id2", how="left")

print("Transaction features added")
```

```
Transaction features added
```

In [9]:
```
event_agg = events.groupby('id2').agg(
    event_count=('id4', 'count'),
    event_unique_offers=('id3', 'nunique'),
    last_event_time=('id4', 'max')
).reset_index()
```

```
train = train.merge(event_agg, on="id2", how="left")
test  = test.merge(event_agg, on="id2", how="left")

print("Event features added")
```

```
Event features added
```

In [10]:
```
offer_summary = offer_meta.groupby('id3').agg(
    offer_discount_mean=('f376', 'mean'),
    offer_redemption_freq=('f375', 'mean'),
    offer_lifespan_days=('id13', lambda x: (x.max() - x.min()).days)
).reset_index()

train = train.merge(offer_summary, on="id3", how="left")
test  = test.merge(offer_summary, on="id3", how="left")

print("Offer metadata features added")
```

```
Offer metadata features added
```

In [11]:
```
train.fillna(-1, inplace=True)
test.fillna(-1, inplace=True)
```

```
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/55000349
9.py:1: FutureWarning: Setting an item of incompatible dtype is deprecated
and will raise an error in a future version of pandas. Value '-1' has dtyp
e incompatible with datetime64[ns], please explicitly cast to a compatible
dtype first.
  train.fillna(-1, inplace=True)
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/55000349
9.py:1: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill,
.bfill is deprecated and will change in a future version. Call result.infe
r_objects(copy=False) instead. To opt-in to the future behavior, set `pd.s
et_option('future.no_silent_downcasting', True)`
  train.fillna(-1, inplace=True)
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/55000349
9.py:2: FutureWarning: Setting an item of incompatible dtype is deprecated
and will raise an error in a future version of pandas. Value '-1' has dtyp
e incompatible with datetime64[ns], please explicitly cast to a compatible
dtype first.
  test.fillna(-1, inplace=True)
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/55000349
9.py:2: FutureWarning: Downcasting object dtype arrays on .fillna, .ffill,
.bfill is deprecated and will change in a future version. Call result.infe
r_objects(copy=False) instead. To opt-in to the future behavior, set `pd.s
et_option('future.no_silent_downcasting', True)`
  test.fillna(-1, inplace=True)
```

In [12]:
```
exclude_cols = ['id1', 'id2', 'id3', 'id4', 'id5', 'y']
features = [col for col in train.columns if col not in exclude_cols]

X = train[features]
y = train['y']
X_test = test[features]
```

In [13]:
```
for df in [X, X_test]:
    for col in df.columns:
        if df[col].dtype == 'object':
            df[col] = pd.to_numeric(df[col], errors='coerce')
```

```
X.fillna(0, inplace=True)
X_test.fillna(0, inplace=True)
```

```
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/217444968
5.py:4: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  df[col] = pd.to_numeric(df[col], errors='coerce')
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/217444968
5.py:6: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  X.fillna(0, inplace=True)
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/217444968
5.py:7: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation: https://pandas.pydata.org/pandas-doc
s/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
  X_test.fillna(0, inplace=True)
```

In [14]:
```
X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

print("Train shape:", X_train.shape)
print("Validation shape:", X_val.shape)
```

```
Train shape: (616131, 377)
Validation shape: (154033, 377)
```

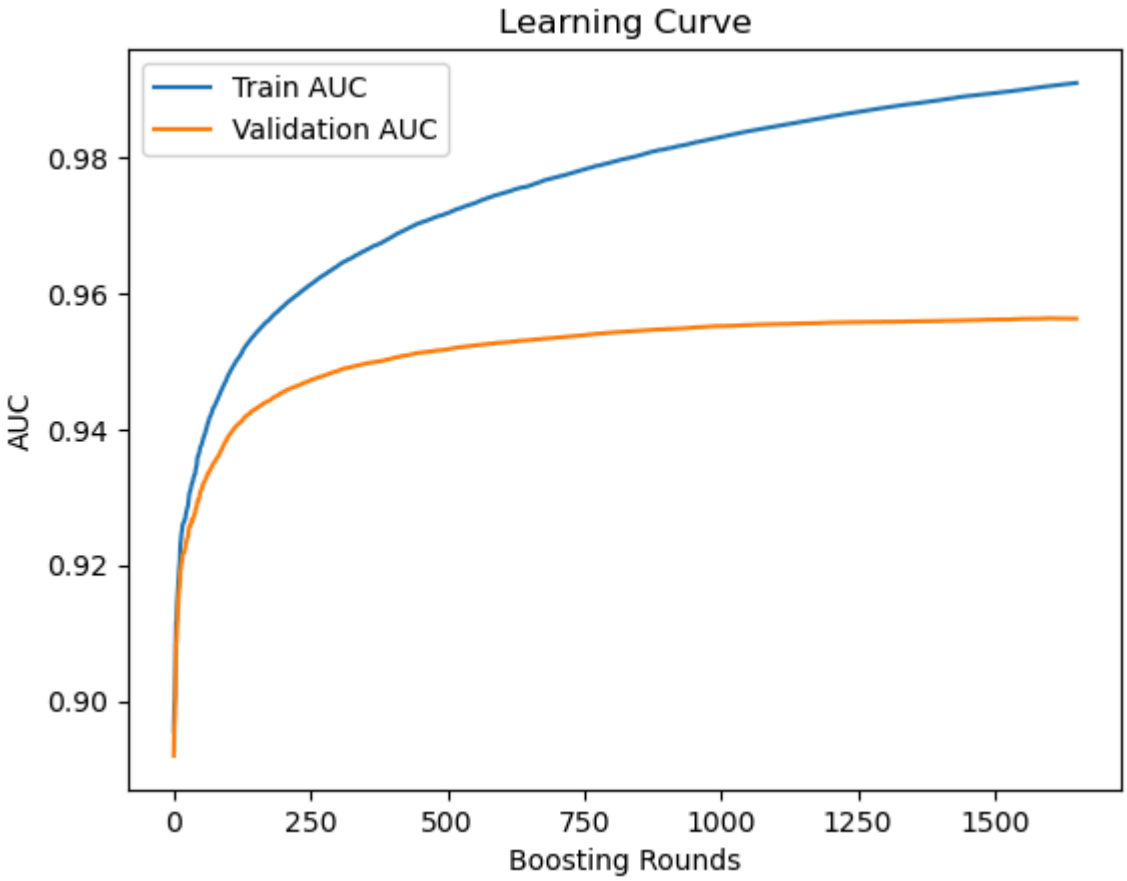
In [15]:
```
dtrain = xgb.DMatrix(X_train, label=y_train)
dval   = xgb.DMatrix(X_val, label=y_val)
dtest  = xgb.DMatrix(X_test)

params = {
    'objective': 'binary:logistic',
    'eval_metric': 'auc',
    'learning_rate': 0.03,
    'max_depth': 7,
    'subsample': 0.9,
    'colsample_bytree': 0.9,
    'min_child_weight': 5,
    'gamma': 0.1,
    'seed': 42,
}

evals_result = {}

model = xgb.train(
    params,
    dtrain,
```

```
    num_boost_round=2000,
    evals=[(dtrain, 'train'), (dval, 'valid')],
    early_stopping_rounds=50,
    evals_result=evals_result,
    verbose_eval=100
)

print("Model trained successfully")
```

```
[0]     train-auc:0.89559       valid-auc:0.89195
[100]   train-auc:0.94801       valid-auc:0.93901
[200]   train-auc:0.95807       valid-auc:0.94545
[300]   train-auc:0.96408       valid-auc:0.94860
[400]   train-auc:0.96838       valid-auc:0.95044
[500]   train-auc:0.97182       valid-auc:0.95178
[600]   train-auc:0.97475       valid-auc:0.95278
[700]   train-auc:0.97716       valid-auc:0.95351
[800]   train-auc:0.97934       valid-auc:0.95427
[900]   train-auc:0.98131       valid-auc:0.95476
[1000]  train-auc:0.98306       valid-auc:0.95520
[1100]  train-auc:0.98460       valid-auc:0.95549
[1200]  train-auc:0.98606       valid-auc:0.95572
[1300]  train-auc:0.98738       valid-auc:0.95586
[1400]  train-auc:0.98847       valid-auc:0.95597
[1500]  train-auc:0.98951       valid-auc:0.95616
[1600]  train-auc:0.99055       valid-auc:0.95635
[1649]  train-auc:0.99100       valid-auc:0.95629
Model trained successfully
```

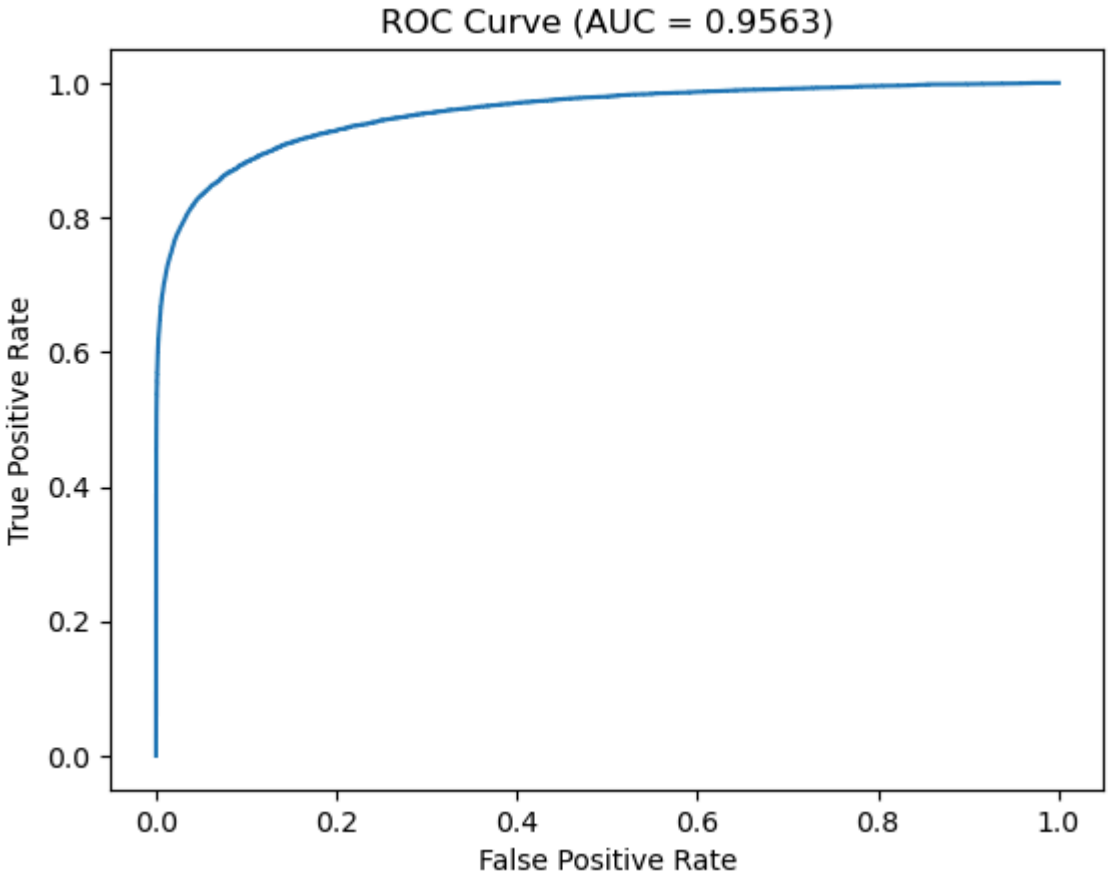
In [19]:

```
train_auc = evals_result['train']['auc']
val_auc = evals_result['valid']['auc']

plt.figure()
plt.plot(train_auc)
plt.plot(val_auc)

plt.xlabel("Boosting Rounds")
plt.ylabel("AUC")
plt.title("Learning Curve")
plt.legend(["Train AUC", "Validation AUC"])

plt.show()
```



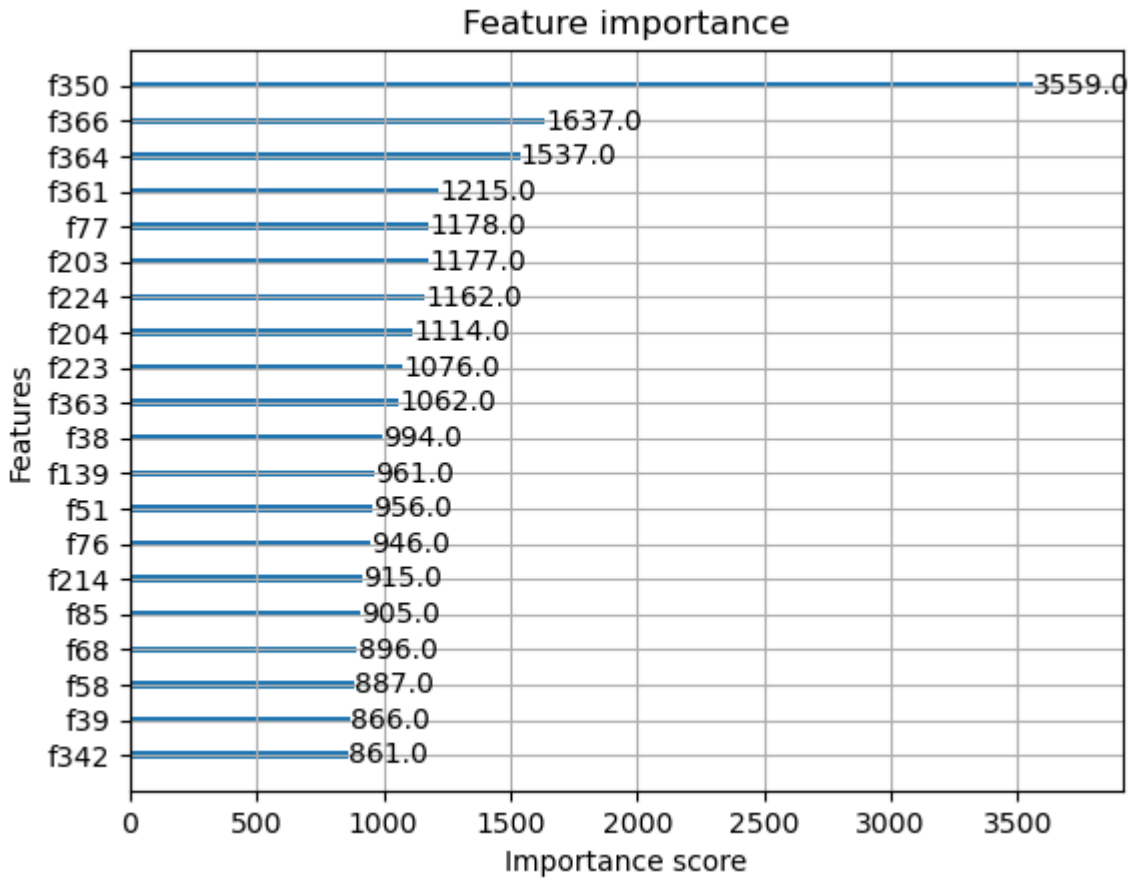
In [23]:
```
from sklearn.metrics import roc_auc_score, roc_curve
```

In [27]:
```
# Ensure numeric labels
y_val = y_val.astype(int)

y_val_pred = model.predict(dval)

fpr, tpr, _ = roc_curve(y_val, y_val_pred)
auc_score = roc_auc_score(y_val, y_val_pred)

plt.figure()
plt.plot(fpr, tpr)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title(f"ROC Curve (AUC = {auc_score:.4f})")

plt.show()

print("Validation AUC:", auc_score)
```




```
Validation AUC: 0.9562939395749248
```

In [29]:
```
xgb.plot_importance(model, max_num_features=20)
plt.show()
```



In [31]:
```
test['pred'] = model.predict(dtest)

submission = test[['id1','id2','id3','id5','pred']]
submission.to_csv("final_submission.csv", index=False)

print("Submission file created.")
```

```
/var/folders/g5/f2ft0ky512zf5csw3s0gmfc00000gn/T/ipykernel_73520/60679722
4.py:1: PerformanceWarning: DataFrame is highly fragmented.  This is usual
ly the result of calling `frame.insert` many times, which has poor perform
ance.  Consider joining all columns at once using pd.concat(axis=1) instea
d. To get a de-fragmented frame, use `newframe = frame.copy()`
  test['pred'] = model.predict(dtest)
Submission file created.
```